

KAVINHQ deployment guide

The production stack contains a FastAPI container and an nginx container. MySQL stays external. Startup can create tables only if missing and can add missing columns after checking `information_schema`. It never creates or drops a database and never replaces an existing admin password. Before changing any table, startup preflights all managed tables. If a populated legacy table is missing an identity or required content column, or its primary key is incompatible, startup stops without applying partial changes and reports that an operator-reviewed migration is required.

Required environment variables

- `DATABASE_URL`: hosted MySQL connection URL, for example `mysql+pymysql://USER:PASSWORD@HOST:3306/DATABASE?charset=utf8mb4`
- `JWT_SECRET`: a private signing key generated with `openssl rand -hex 32`
- `ADMIN_EMAIL`: first admin email, used only when the admin table is empty
- `ADMIN_PASSWORD`: first admin password, used only when the admin table is empty
- `FRONTEND_URL`: canonical public origin such as `https://kavinhq.com`
- `CORS_ORIGINS`: comma-separated allowed browser origins
- `TRUST_PROXY_HEADERS`: set to `true` behind Coolify's HTTPS proxy

Never commit a populated `.env` file. The example values are placeholders, not working credentials.

Coolify

1. Create a new Docker Compose resource from this repository.
2. Use `docker-compose.yml` and expose the frontend service on port 80.
3. Add every required environment variable in Coolify. Keep `VITE_API_BASE` empty because nginx forwards relative `/api` requests to the backend container.
4. Attach the public domain to the frontend service and enable HTTPS.
5. Deploy, then check `https://YOUR_DOMAIN/api/health`. A healthy response reports both the application and database as available.
6. Sign in once at `https://YOUR_DOMAIN/admin` and replace all seeded sample portfolio content as needed.

The frontend image is built with Node 20 and served by nginx. Static assets receive a 30-day cache. HTML routes fall back to `index.html`, and `/api/` is proxied to the backend service.

Local admin

The local admin compose file connects to the same hosted MySQL database while exposing the site only on your machine.

1. Copy `.env.example` to `.env`.
2. Set `FRONTEND_URL=http://localhost:8081`, `CORS_ORIGINS=http://localhost:8081`, and `TRUST_PROXY_HEADERS=false`.
3. Run `docker compose -f docker-compose.admin.yml up --build`.

4. Open `http://localhost:8081/admin`. The API is also available at `http://localhost:8001/api/health` for diagnostics.

Stop the local stack with `docker compose -f docker-compose.admin.yml down`. This removes containers and the internal Docker network only. It does not remove hosted MySQL data.

MySQL TLS

Use the TLS settings supplied by the database host. PyMySQL accepts URL query parameters. A provider URL may look like `mysql+pymysql://USER:PASSWORD@HOST:3306/DATABASE?charset=utf8mb4&ssl_ca=%2Fpath%2Fto%2Fca.pem`. Mount the CA file into the backend container when the provider requires a custom certificate. Prefer a provider URL that verifies the server certificate.

Security and operations

- Keep MySQL private. Allow inbound connections only from the Coolify host and trusted administrator machines.
- Use a least-privilege database user limited to the configured schema and the table operations this application needs.
- Use a unique, long admin password and rotate `JWT_SECRET` through a planned sign-out window.
- Keep HTTPS enabled. Authentication cookies use secure cross-site settings when requests arrive through HTTPS.
- Configure automated MySQL backups with the hosting provider and test restoration. Docker Compose does not back up the external database.
- Review `/api/health` after releases. Do not expose database credentials in logs, build arguments, or image layers.
- Update base images and Python or npm packages on a regular maintenance schedule.

Non-destructive startup

On startup the backend checks its required tables and columns. It seeds the initial admin only when `admin_users` is empty, six replaceable projects only when `projects` is empty, and default WhatsApp settings only when `settings` is empty. Existing records and passwords are preserved.

Release verification

Run the automated API check only against a database where creating and deleting its temporary `verification-project` record is acceptable. Set `KAVINHQ_API_BASE`, `ADMIN_EMAIL`, and `ADMIN_PASSWORD`, then run `powershell -ExecutionPolicy Bypass -File scripts/verify-api.ps1`. The script restores the prior WhatsApp settings and removes its temporary project in a cleanup block.

For visual checks, use a database where temporary project creation and deletion are acceptable. Set `KAVINHQ_SITE_BASE` to the running frontend and run `node scripts/verify-ui.mjs`. Cleanup uses the frontend's `/api` proxy by default. Set `KAVINHQ_API_BASE` only when a development frontend does not expose that proxy. The script verifies project creation, editing, confirmed deletion, settings save, logout, public pages, the missing-project state, contact preview, and reduced motion at both required viewports.

Cleanup failures are reported. Use `KAVINHQ_VIEWPORT=desktop` or `KAVINHQ_VIEWPORT=mobile` for a targeted rerun.